

TRAJCENTER v2.0 — Protocole RWS PC ↔ Robot

Version : 2.5 draft

Date : 2026-08-26

RobotWare : 6.x

Transport : ABB Robot Web Services uniquement

TCP custom / watchdog / polling nominal : supprimés

Module RAPID : module système unique TRAJCENTER

1. Module RAPID système

TrajCenter v2.0 utilise un **module système RAPID unique** :

```
MODULE TRAJCENTER(SYSMODULE)
```

Fichier attendu :

```
rapid/TRAJCENTER.mod
```

Ce module regroupe tout le protocole robot côté ABB :

Section interne	Rôle
Section 1	Types RECORD
Section 2	Limites globales
Section 3	Codes d'état et d'erreur
Section 4	Types de mouvement
Section 5	Process : constantes + catalogue
Section 6	Configuration cellule : tools, wobjs
Section 7	Variables RWS PC ↔ robot
Section 8	Defaults robot

Il n'y a plus d'ordre de chargement entre plusieurs modules TrajCenter.

Le PC doit cibler le module RAPID unique :

```
TRAJCENTER
```

Exemples de symboles RWS :

```
TRAJCENTER/sendTrajRequest
TRAJCENTER/refreshMetaRequest
TRAJCENTER/trajTools
TRAJCENTER/trajWobjjs
TRAJCENTER/processTypes
TRAJCENTER/trajData{1}
```

Politique :

Cas	Déclaration
Flags abonnés RWS	PERS
Tools / wobjs cellule	PERS
Maintenance tools / wobjs	PERS
Runtime, metadata, trajectoire, defaults	VAR
Tailles fixes, codes	CONST

Politique de modification :

Section	Politique
Types RECORD	Ne pas modifier sauf évolution coordonnée PC/RAPID
Limites globales	Ne pas modifier sauf changement volontaire de capacité
Codes d'état/erreur	Ne pas modifier sans mise à jour PC + documentation
Types de mouvement	Ne pas modifier sauf ajout officiel supporté
Process	Modifier uniquement ici pour ajouter/retirer/renommer un process
Configuration cellule	Modifiable par le programmeur robot
Variables RWS	Ne pas modifier noms/types/dimensions sans mise à jour PC
Defaults robot	Réglables à la mise en service

2. Flux

Flux	Déclencheur	Direction	Mécanisme
Refresh metadata	refreshMetaRequest = TRUE	Robot → PC	RWS subscription
Envoi trajectoire	sendTrajRequest = TRUE	Robot → PC	RWS subscription
Écriture données	PC	PC → Robot	RWS write + Mastership
Lecture contexte	PC	Robot → PC	RWS read

Règles :

PC subscribe:

- TRAJCENTER/sendTrajRequest
- TRAJCENTER/refreshMetaRequest

PC subscribe (via `abb_rws_client_python_rw6.highlevel.subscription`):

- resource: `symbol("T_ROB1", "TRAJCENTER", "sendTrajRequest")`
priority: "1" (Medium) – PERS bool, latence 200ms
- resource: `symbol("T_ROB1", "TRAJCENTER", "refreshMetaRequest")`
priority: "1" (Medium) – PERS bool, latence 200ms

Seuls les events TRUE déclenchent une action.

Les events FALSE sont ignorés.

Le PC remet la requête traitée à FALSE, succès comme erreur.

Au démarrage/reconnexion, relire les deux flags pour traiter une demande pendante.

Toute écriture RWS est faite sous Mastership avec release garanti.

3. Constantes et limites

Module : **TRAJCENTER**

3.1 Limites globales

```
CONST num maxTrajCount := 256;
CONST num maxTrajPointCount := 100000;
CONST num maxProcessParamSetCount := 256;
CONST num maxProcessParamPerSet := 10;
```

3.2 Types de mouvement

```
CONST num moveTypeL := 0;
CONST num moveTypeJ := 1;
CONST num moveTypeC := 2;
```

3.3 Process

```
CONST num processNone := 0;
CONST num processAcf := 1;
CONST num processAak := 2;
CONST num processPushcorp := 3;
```

3.4 Codes de succès

```
CONST num statusOk := 200000;  
CONST num statusMetadataRefreshed := 200001;  
CONST num statusTrajectoryTransferred := 200002;
```

Les codes d'erreur complets sont listés en section 12.

4. Types RAPID

4.1 trajCenterPointData

```
RECORD trajCenterPointData  
  num moveType;  
  robtarget point;  
  num tcpSpeed;  
  num zoneType;  
  bool readConfs;  
  num toolIndex;  
  num wobjIndex;  
  num processParamIndex;  
ENDRECORD
```

Champ	Convention
moveType	0=MoveL, 1=MoveJ, 2=MoveC
point	robtarget ABB
tcpSpeed	mm/s, > 0
zoneType	zone autorisée, 255=fine
readConfs	prise en compte confdata
toolIndex	base 1 dans trajTools, 0=undefined
wobjIndex	base 1 dans trajWobjs, 0=undefined
processParamIndex	base 1 dans processParams, 0=aucun

4.2 trajCenterTrajMeta

```
RECORD trajCenterTrajMeta  
  string name;  
  num pointCount;  
  num processType;  
ENDRECORD
```

Champ	Convention
name	nom affichable
pointCount	nombre de points
processType	0=NONE, 1=ACF, 2=AAK, 3=PUSHCORP, 4..255=RESERVED

4.3 Tools / wobjjs

```

RECORD trajCenterTool
  string name;
  tooldata value;
ENDRECORD

```

```

RECORD trajCenterWobj
  string name;
  wobjdata value;
ENDRECORD

```

Mapping PC :

```

tool_name -> trajTools{i}.name -> toolIndex = i
wobj_name -> trajWobjjs{i}.name -> wobjIndex = i

```

4.4 Process

```

RECORD trajCenterProcessParameter
  string name;
  num value;
ENDRECORD

```

```

RECORD trajCenterProcessType
  num id;
  string name;
ENDRECORD

```

Convention :

```

processParams{i,j}.name = "" => slot inutilisé
processParams{i,j}.value    => numérique uniquement

```

5. Catalogue process

Module : **TRAJCENTER**

```
CONST num processNone := 0;
CONST num processAcf := 1;
CONST num processAak := 2;
CONST num processPushcorp := 3;

CONST num processTypeCount := 4;

VAR trajCenterProcessType processTypes{processTypeCount}:=[
  [processNone, "NONE"],
  [processAcf, "ACF"],
  [processAak, "AAK"],
  [processPushcorp, "PUSHCORP"]
];
```

ID	Nom
0	NONE
1	ACF
2	AAK
3	PUSHCORP
4..255	RESERVED

Pour ajouter un process :

1. Ajouter une constante process dans la section PROCESS.
2. Incrémenter processTypeCount.
3. Ajouter l'entrée correspondante à processTypes.
4. Mettre à jour la documentation et le client PC si nécessaire.

6. Configuration cellule

Module : **TRAJCENTER**

```
PERS trajCenterTool trajTools{N};
PERS trajCenterWobj trajWobjs{M};

PERS tooldata tempTool;
PERS wobjdata tempWobj;
```

Règles :

```
trajTools/trajWobjjs sont cellule-dépendants.
Le PC lit .name.
Index RAPID envoyés en base 1.
0 = non défini.
```

Les fichiers **.trajcenter** ne contiennent pas les **tooldata** ou **wobjdata** complets. Ils utilisent des noms logiques :

```
tool_name
wobj_name
```

Le PC résout ces noms en index via **trajTools** et **trajWobjjs**.

7. Variables RWS

Module : **TRAJCENTER**

7.1 Requêtes

```
PERS bool sendTrajRequest := FALSE;
PERS bool refreshMetaRequest := FALSE;
VAR num selectedTrajIndex := 0;
```

```
selectedTrajIndex = 0 : aucune sélection
selectedTrajIndex = 1..nbTrajAvailable: trajectoire valide
```

7.2 État

```
VAR bool trajReady := FALSE;
VAR bool transferError := FALSE;
VAR num lastErrorCode := statusOk;
VAR string lastError := "";
VAR num transferProgress := 0;
```

Variable	Rôle
trajReady	trajectoire complète et exécutable

Variable	Rôle
<code>transferError</code>	dernier refresh/transfert en erreur
<code>lastErrorCode</code>	code état/erreur
<code>lastError</code>	message court
<code>transferProgress</code>	<code>0..100</code>

7.3 Metadata store

```
VAR num nbTrajAvailable := 0;
VAR trajCenterTrajMeta trajectories{256};
```

Entrées valides :

```
trajectories{1..nbTrajAvailable}
```

7.4 Trajectoire chargée

```
VAR num nbLoadedTrajPoints := 0;
VAR trajCenterPointData trajData{100000};
VAR trajCenterProcessParameter processParams{256,10};
```

Entrées valides :

```
trajData{1..nbLoadedTrajPoints}
processParams{1..256,1..10}
```

Aucun process :

```
trajData{i}.processParamIndex = 0
```

Process associé :

```
trajData{i}.processParamIndex = p
=> processParams{p,1..10}
```

7.5 Defaults robot

```
VAR bool hasDefaultTcpSpeed := FALSE;
VAR num defaultTcpSpeed := 0;

VAR bool hasDefaultZoneType := FALSE;
VAR num defaultZoneType := 255;

VAR bool hasDefaultToolName := FALSE;
VAR string defaultToolName := "";

VAR bool hasDefaultWobjName := FALSE;
VAR string defaultWobjName := "";

VAR num defaultMoveType := moveTypeL;
VAR bool defaultReadConfs := TRUE;
```

Règle :

Le PC ne doit jamais inventer silencieusement tool, wobj, speed ou zone.
Fallback uniquement si hasDefault* = TRUE.

Pour tool et wobj :

```
defaultToolName -> résolution dans trajTools -> toolIndex
defaultWobjName -> résolution dans trajWobjs -> wobjIndex
```

8. Format .trajcenter

Archive :

```
meta.json
points.parquet
```

8.1 meta.json

Champs usuels :

Champ	Type	Rôle
name	str	nom trajectoire
version	str	version format

Champ	Type	Rôle
source_format	str	format source
robot_model	str null	robot cible optionnel
process_type	str int null	process principal optionnel
extra	dict	metadata libre

process_type :

```
absent/null/"NONE"/0 -> 0
"ACF"/1               -> 1
"AAK"/2               -> 2
"PUSHCORP"/3          -> 3
autre                  -> validation contre processTypes robot
```

8.2 Colonnes points.parquet

Colonne	Exportable	Envoyable	Résolution
x	oui	oui	obligatoire
y	oui	oui	obligatoire
z	oui	oui	obligatoire
q1	oui	oui	quaternion ABB w
q2	oui	oui	quaternion ABB x
q3	oui	oui	quaternion ABB y
q4	oui	oui	quaternion ABB z
cf1	non	oui	0 si absent
cf4	non	oui	0 si absent
cf6	non	oui	0 si absent
cfx	non	oui	0 si absent
eax_a..eax_f	non	oui	absent/NaN = 9E+9 à l'écriture RWS uniquement
tcp_speed	non	oui	erreur sauf default robot
zone_type	non	oui	erreur sauf default robot
move_type	non	oui	default robot si absent
tool_name	non	oui	erreur sauf default robot validé
wobj_name	non	oui	erreur sauf default robot validé

Colonne	Exportable	Envoyable	Résolution
<code>readconfs</code>	non	oui	règle ci-dessous
<code>process_type</code>	non	oui	optionnel, sinon <code>meta.process_type</code> , sinon <code>NONE</code>
<code>process_params</code>	non	oui	optionnel
<code>process_param_index</code>	non	non	généré PC, non fiable si stocké

Règle `readconfs` si absent :

```
si cf1/cf4/cf6/cfx présents : readConfs = TRUE
sinon                        : readConfs = FALSE
```

Règle process :

```
process_type absent partout -> processType = 0 = NONE
process_params vide/absent  -> processParamIndex = 0
process_param_index stocké  -> ignoré et recalculé
```

8.3 `process_params`

Formats acceptés :

```
{"force": 120.0, "feed": 5.0}
```

ou JSON string :

```
{ "force": 120.0, "feed": 5.0 }
```

Valeurs vides :

```
None, NaN, "", {} => aucun paramètre
```

Contraintes :

```
sets distincts <= 256
paramètres par set <= 10
noms non vides
valeurs numériques uniquement
```

```
sets identiques dédoublés
ordre paramètres = tri alphabétique par nom
```

8.4 Exportable vs envoyable

Niveau	Définition
Exportable	<code>x, y, z, q1, q2, q3, q4</code> présents
Envoyable	champs robot résolus + tools/wobjjs/process/defaults valides

Une trajectoire exportable peut être listée mais refusée à l'envoi.

9. Conventions

9.1 Indexation

Élément	Convention
Tableaux RAPID	base 1
<code>selectedTrajIndex</code>	base 1, <code>0=aucune sélection</code>
<code>toolIndex, wobjIndex</code>	base 1, <code>0=undefined</code>
<code>processParamIndex</code>	base 1, <code>0=aucun</code>
Points <code>.trajcenter</code>	ordre fichier -> RAPID <code>{1..N}</code>

9.2 RWS symbol URLs

RAPID array :

```
trajData{1}
```

RWS symbolurl :

```
RAPID/T_ROB1/TRAJCENTER/trajData%7B1%7D
```

Record complet recommandé :

```
trajData{1} = [moveType, robtarget, tcpSpeed, zoneType, readConfs, toolIndex,
wobjIndex, processParamIndex]
```

9.3 Robtarget

Format RWS :

```
[ [x,y,z], [q1,q2,q3,q4], [cf1,cf4,cf6,cfx], [eax_a,eax_b,eax_c,eax_d,eax_e,eax_f]]
```

Règles :

```
[q1,q2,q3,q4] = [w,x,y,z]
axes externes absents/NaN -> 9E+9 à l'écriture RWS uniquement
9E+9 jamais stocké dans .trajcenter
```

9.4 Zones

Autorisées :

```
0, 1, 5, 10, 15, 20, 30, 40, 50, 60, 80, 100, 150, 200, 255
```

```
0    = z0
255 = fine
```

9.5 Vitesses

```
tcp_speed -> trajData[i].tcpSpeed
numérique, mm/s, > 0
```

9.6 Mouvements

RAPID	Mouvement	Alias acceptés
0	MoveL	"L", "MoveL", 0
1	MoveJ	"J", "MoveJ", 1
2	MoveC	"C", "MoveC", 2

9.7 MoveC

Encodage :

```
MoveC = deux points consécutifs C,C  
C,C,C,C valide  
C,C,C invalide
```

Pour chaque paire C,C, doivent être identiques :

```
tcpSpeed  
toolIndex  
wobjIndex  
zoneType  
readConfs  
processParamIndex
```

10. Pipelines

10.1 Démarrage PC

1. Ouvrir session RWS.
2. S'abonner à sendTrajRequest et refreshMetaRequest.
3. Lire l'état courant des deux requêtes.
4. Lire defaults robot.
5. Lire trajTools et trajWobjs.
6. Lire processTypes.
7. Scanner trajectory_store/.
8. Écrire nbTrajAvailable et trajectories.
9. Traiter requête pendant si TRUE.

10.2 Refresh metadata

Déclencheur :

```
refreshMetaRequest := TRUE;
```

PC :

1. Reçoit refreshMetaRequest = TRUE.
2. Scanne trajectory_store/.
3. Liste les .trajcenter exportables.
4. Détermine name, pointCount, processType.
5. Écrit nbTrajAvailable.
6. Écrit trajectories{1..nbTrajAvailable}.

7. Écrit `transferError = FALSE`.
8. Écrit `lastErrorCode = 200001`.
9. Écrit `lastError = ""`.
10. Écrit `refreshMetaRequest = FALSE`.

`trajectories{i}`:

[name, pointCount, processType]

Résolution `processType` metadata :

1. `meta.process_type` si présent
2. sinon `process_type` points si unique non-NONE
3. sinon premier process non-NONE
4. sinon 0 = NONE

10.3 Envoi trajectoire

Déclencheur :

```
selectedTrajIndex := k;
sendTrajRequest := TRUE;
```

PC :

1. Reçoit `sendTrajRequest = TRUE`.
2. Lit `selectedTrajIndex`.
3. Charge le `.trajcenter`.
4. Lit `defaults`, `trajTools`, `trajWobjs`, `processTypes`.
5. Résout `toolIndex`, `wobjIndex`, `processType`, `processParams`.
6. Valide limites et contraintes.
7. Écrit `trajReady = FALSE`.
8. Écrit `transferError = FALSE`.
9. Écrit `transferProgress = 0`.
10. Écrit `nbLoadedTrajPoints`.
11. Écrit `processParams` si nécessaire.
12. Écrit `trajData{1..nbLoadedTrajPoints}`.
13. Met à jour `transferProgress` par blocs.
14. Écrit `transferProgress = 100`.
15. Écrit `lastErrorCode = 200002`.
16. Écrit `lastError = ""`.
17. Écrit `trajReady = TRUE`.
18. Écrit `sendTrajRequest = FALSE`.

En erreur :

```
trajReady = FALSE
transferError = TRUE
lastErrorCode = code
lastError = message court
transferProgress = valeur courante ou 0
sendTrajRequest = FALSE
refreshMetaRequest = FALSE si erreur refresh
```

11. Validation avant envoi

Refus si :

Cas	Code
<code>selectedTrajIndex</code> hors bornes	400001
fichier absent	400002
format <code>.trajcenter</code> invalide	400003
<code>pointCount</code> > <code>maxTrajPointCount</code>	400004
<code>zone_type</code> invalide	400005
<code>move_type</code> invalide	400006
paire <code>MoveC</code> invalide	400007
<code>tcp_speed</code> absent sans default	400008
<code>zone_type</code> absent sans default	400009
<code>tool_name</code> absent sans default	400010
<code>wobj_name</code> absent sans default	400011
<code>tool_name</code> introuvable	400012
<code>wobj_name</code> introuvable	400013
<code>tcp_speed</code> <= 0	400014
<code>readconfs</code> invalide	400015
robtarget non sérialisable	400016
process inconnu	400017
trop de sets process	400018
process params invalides	400019

Process params invalides :

JSON invalide
nom vide
valeur non numérique
plus de 10 paramètres par set
plus de 256 sets distincts

12. Codes d'état et d'erreur

Code	Signification
200000	OK
200001	Metadata refreshed
200002	Trajectory transferred
400001	<code>selectedTrajIndex</code> hors bornes
400002	Fichier trajectoire introuvable
400003	Format <code>.trajcenter</code> invalide
400004	Trop de points
400005	<code>zone_type</code> invalide
400006	<code>move_type</code> invalide
400007	Paire <code>MoveC</code> invalide
400008	<code>tcp_speed</code> manquant sans default
400009	<code>zone_type</code> manquant sans default
400010	<code>tool_name</code> manquant sans default
400011	<code>wobj_name</code> manquant sans default
400012	<code>tool_name</code> introuvable côté robot
400013	<code>wobj_name</code> introuvable côté robot
400014	Vitesse invalide
400015	<code>readConfs</code> invalide
400016	Robtarget invalide
400017	Process inconnu
400018	Trop de sets process
400019	Paramètres process invalides
401001	Authentification RWS refusée

Code	Signification
403001	Mastership refusé
403002	Écriture RWS interdite
404001	Symbole RAPID introuvable
404002	<code>trajTools</code> introuvable
404003	<code>trajWobjs</code> introuvable
404004	Store trajectoire introuvable
404005	Default robot introuvable
404006	<code>processTypes</code> introuvable
408001	Timeout requête RWS
408002	Timeout transfert
409001	Transfert déjà en cours
409002	État robot incompatible
500001	Erreur interne client
500002	Erreur de sérialisation
500003	Erreur de conversion trajectoire
502001	Réponse RWS invalide
503001	Contrôleur indisponible
504001	Timeout contrôleur